Ограниченный контекст — не ограниченная информация: структурированное состояние обменивает способность решать задачи на размер промпта в кодирующих агентах с длинным горизонтом

Alex Markov-Buturskiy

Draft 21 сентября 2026 г.

Аннотация

Структурированное состояние заменяет историю диалога агента компактным, поддерживаемым моделью блоком состояния. На синтетических бенчмарках с длинным горизонтом это ограничивает рост промпта почти постоянным размером и не сообщает о потере качества выполнения задач. Мы проверяем, переносится ли этот результат на реальную работу с кодом, в закрытом дизайне с предрегистрацией: 87 задач Terminal-Bench 2.0 $\times$ 4 условий $\times$ 3 независимых испытаний = 1044 запусков, каждая ячейка заполнена, вердикты берутся только из собственного верификатора каждой задачи.

В этом режиме перенос не происходит. Метод в опубликованном виде решает 0.349 задач против 0.613 у контрольной группы с транскриптом; при попарном сравнении по (задаче, испытанию) он проигрывает 82 задач и выигрывает 13 ($p = 2.1 \times 10^{-13}$). Потеря больше на задачах с длинным горизонтом (54 против 8, $p = 1.7 \times 10^{-9}$), чем на коротких, что противоположно тому, ради чего задуман механизм памяти. Восстановление шести дословных обменов рядом с блоком состояния не оставляет обнаружимого дефицита (29 против 23 несовпадающих задач, $p = 0.49$), что локализует ущерб в отброшенной истории, а не в самом блоке состояния. Мы сообщаем об этом как о неспособности обнаружить различие, а не как о доказанной эквивалентности.

Запуски показывают механизм. Ограничение промпта не ограничивает приобретение информации: агенты заново получают отброшенный контекст через свои инструменты, повторяя побайтово идентичные команды осмотра в 58 % всех команд в неуспешных запусках только-состояния против 20 % у контрольной группы, и переходя по указателям на файлы в 12 раз чаще, так что цикл инструментов тратит на ходы то, что промпт сэкономил на контексте. Экономика следует из этого: при равном исходе рука только-состояния несёт 0.47$\times$ от пикового промпта контрольной группы, но стоит 2.57$\times$ дороже, потому что поддержание состояния тарифицируется по выходному тарифу (4.55$\times$), а блок состояния, переписываемый на каждом ходу, разрушает кэш префикса, которым пользуется контрольная группа (352.2M кэшированных токенов против 6.2M).

Проверенный режим — одно семейство моделей на одном бенчмарке, и это режим, в котором контрольная группа никогда не приближается к

пределу контекста, так что эти результаты ограничивают то, что даёт механизм, когда окно не является связывающим ограничением, а не то, что он даёт, когда оно им является.

Наш вклад состоит в фальсификации утверждения о переносимости на реальных задачах с кодом, выявлении свойства оценки, которое скрывало это (фиксация числа обращений к модели за эпизод делает компенсаторное повторное приобретение невозможным по построению), методе учёта затрат с учётом кэша, при котором сокращение токенов и сокращение затрат показаны как разные величины, и опубликованном наборе данных из 1044 полностью инструментированных траекторий агента.

Как читать пометки источника. Утверждения о нашей собственной системе несут пометку ^{live} для измерения на реальном запуске, ^{code} для свойства кода и ^{orig} для утверждения из исходных статей. Всё, что без пометки, — это аргумент, а не свидетельство.

1 Введение

Кодирующий агент, работающий часами, накапливает транскрипт, который растёт без ограничений. Каждое наблюдение, каждый результат работы инструмента и каждая промежуточная мысль отправляются заново при следующем обращении к модели, так что промпт растёт вместе с задачей, а стоимость хода растёт вместе со всем, что ему предшествовало. Структурированное состояние — наиболее прямой ответ на это: отбросить транскрипт и заставить модель поддерживать компактное структурированное состояние, которое должно быть достаточной статистикой для того, что ещё предстоит сделать. skill.state [Badhe et al., 2026] сообщает, что это удерживает размер промпта почти постоянным от 10 до 200 событий среды, при этом качество не ухудшается^{orig}.

Этот результат измерен на синтетическом генераторе. Данная статья задаётся вопросом, переживает ли он контакт с реальными программными задачами, где агент сам решает, что осматривать и когда работа завершена.

Утверждение, которое мы проверяем. Не «полезно ли структурированное состояние» в общем смысле, а конкретное утверждение о переносимости, следующее из опубликованной оценки: что замена транскрипта поддерживаемым блоком состояния сохраняет способность решать задачи при ограничении контекста. Мы проверяем это на Terminal-Bench 2.0, с дизайном, зафиксированным письменно до какого-либо запуска (§4), и вердиктами, взятыми только из собственного верификатора каждой задачи.

Результат. Перенос не происходит. На 1044 запусках опубликованная конфигурация решает 0.349 задач против 0.613 у контрольной группы с транскриптом, проигрывая 82 парных задач и выигрывая 13 ($p = 2.1 \times 10^{-13}$). Это не артефакт трудного корпуса или слабой модели: та же модель на тех же задачах с неповреждённой историей их решает.

Два дальнейших наблюдения превращают это из отрицательного результата в объяснение. Во-первых, ущерб растёт с горизонтом задачи (§7), что в точности инвертирует аргумент в пользу метода, чья выгода предположительно должна накапливаться на длинных задачах. Во-вторых, восстановление всего

шести дословных обменов рядом с блоком состояния оставляет дефицит, который мы больше не можем обнаружить ($p = 0.49$; отсутствие свидетельства, а не свидетельство эквивалентности), что помещает потерю в отброшенную историю, а не в абстракцию состояния.

Почему исходная оценка не могла этого увидеть. В синтетической постановке число обращений к модели за эпизод фиксировано по построению: среда порождает T событий, и каждый рантайм отвечает на каждое ровно одним обращением^{orig}. Агент, что-то забывший, поэтому не может заплатить за восстановление этого: в дизайне нет механизма, посредством которого повторное приобретение могло бы проявиться как затрата. В агентной постановке число шагов — это результат, а не управляющий параметр, и именно там появляется цена (§6).

Механизм, сформулированный как тезис статьи.

> Ограниченный конверсационный контекст не подразумевает ограниченного эффективного приобретения информации, потому что кодирующие агенты компенсируют это повторным чтением среды.

Блок состояния ограничивает то, что агенту сообщают; он не ограничивает то, что агенту нужно. Разница оплачивается через цикл инструментов, который не наблюдает ни одна метрика размера промпта. Мы измеряем это напрямую: в неуспешных запусках только-состояния 58 % всех команд являются повторами уже выполненного осмотра, против 20 % у контрольной группы^{live} (§8).

Экономика следует из того же механизма. Поскольку поддержание состояния выдаётся как сгенерированный текст, оно тарифицируется по выходному тарифу; поскольку блок переписывается на каждом ходу, он разрушает кэш префикса, из которого почти полностью обслуживается только дополняемый транскрипт. При равном исходе рука только-состояния несёт $0.47\times$ от пикового промпта контрольной группы и стоит $2.57\times$ дороже^{live}. Сокращение числа токенов — не сокращение затрат, и на этом корпусе это даже не сокращение затрат в том направлении, на которое указывает число токенов.

Вклад.

1. Фальсификация, на закрытом предрегистрированном дизайне из 1044 запусков, утверждения о том, что структурированное состояние сохраняет способность решать задачи на реальных программных задачах (§7).
2. Выявление свойства оценки, которое это скрывало: фиксация числа обращений к модели за эпизод делает компенсаторное повторное приобретение неизмеримым по построению (§6).
3. Механизм, измеренный на уровне отдельных вызовов инструментов, связывающий отброшенную историю с избыточным осмотром среды (§8).
4. Методология учёта затрат с учётом кэша, при которой сокращение токенов и денежное сокращение показаны как различные величины, и при которой ни одна конфигурация структурированного состояния не дешевле транскрипта при равном исходе (§9).

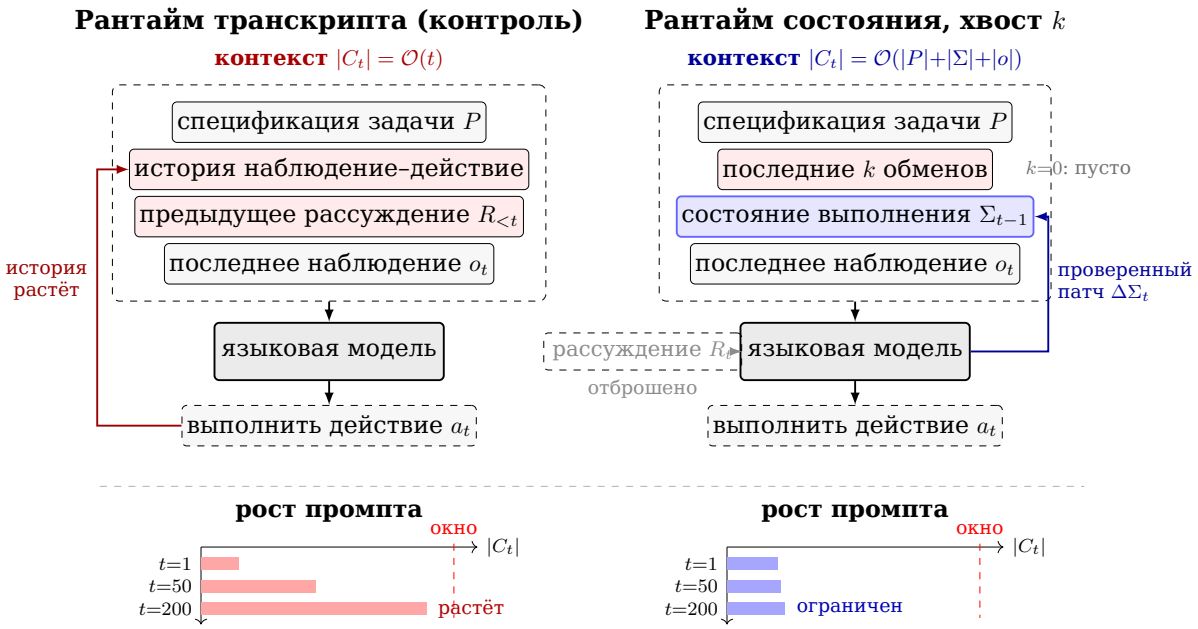


Рис. 1: Две политики контекста из Алгоритмов 1 и 2. **Слева:** рантайм транскрипта дополняет каждый обмен, так что промпт растёт вместе с эпизодом (Ур. 1) и в итоге упирается в окно контекста. **Справа:** рантайм состояния несёт проверенный блок состояния и, при k=0, вообще никакой истории (Ур. 3); его промпт ограничен независимо от t. Оба отбрасывают промежуточное рассуждение. Ограниченный промпт — это опубликованная выгода. Чего диаграмма не может показать — это величина, которую измеряет данная статья: ограничение $|C_t|$ не ограничивает информацию, требуемую задачей, так что агент заново её получает через вызовы инструментов, раздувая реализованный горизонт (Ур. 6) без затрат, видимых какой-либо метрике размера промпта.

5. Опубликованный набор данных из 1044 полностью инструментированных траекторий с происхождением по каждому запуску, а также детерминированный анализ, воспроизводящий каждое число в этой статье (§A).

Что мы не утверждаем. Мы не утверждаем, что структурированное состояние — плохая идея. Мы утверждаем, что его опубликованная оценка измеряет режим, в котором его центральный режим отказа не может возникнуть, и что на реальных программных задачах механизм обменивает способность решать задачи на размер промпта по ставке, которую исходная метрика не может выразить. Раздел 11 прямо называет границы: исследование охватывает одно семейство моделей, один корпус и исследование композиции, которое всё ещё продолжается.

2 Контекст

2.1 Память со структурированным состоянием

skill.state [Badhe et al., 2026] поддерживает устойчивое структурированное состояние Σ_t и собирает каждый промпт как

$$\text{промпт}_t = [\text{спецификация задачи}] + [\Sigma_t] + [o_t],$$

где o_t — последнее наблюдение, а история взаимодействия отбрасывается. Модель выдаёт рассуждение, патч состояния и действие за одну генерацию, $(R_t, \Delta\Sigma_t, a_t)$, как один объект JSON ровно с двумя ключами `state_patch` и `action`[orig]. Рантайм проверяет каждый патч по схеме и отклоняет недействительные с циклом отката и повтора. Схемы пишутся один раз на домен; на 100 экземплярах InterCode CTF повторно используется одна пятиполевая схема[orig].

Приводимое доказательство — это развёртка по горизонту на процедурном генераторе: размер промпта остаётся около 1800 символов от $T{=}10$ до $T{=}200$, тогда как базовые линии растут до 48,000, а качество сохраняется или улучшается[orig]. Собственная таксономия ошибок статьи для моделей с открытыми весами приписывает 68 % отказов преждевременной перезаписи или удалению состояния[orig]— цифра, к которой мы вернёмся в §10, потому что это аргумент в пользу вывода части состояния из-под контроля модели.

2.2 Детерминированное состояние выполнения

ledger [Wang et al., 2026] занимает противоположный угол дизайна: поддерживаемая рантаймом запись, механически выводимая из завершённых взаимодействий: записи наблюдений, счётчики изменений по файлам и глобальные, нормализованные записи команд с их исходами. Он не добавляет обращений к модели и не изменяет агента. Его два механизма аккуратно разделяются в его собственной абляции: inform (отрисовка записи в конце входа, где она не может нарушить дополняемый префикс) даёт основную часть выигрыша по эффективности, тогда как govern (перехват предложенной команды и повторное использование более раннего результата) даёт основную часть выигрыша по разрешению[orig].

Две системы дополняют, а не соревнуются друг с другом: skill.state удерживает семантическое состояние в модели, тогда как ledger удерживает объективное состояние в рантайме. Наше исследование композиции (§10) варьирует именно эту границу, и мы намеренно исключаем из него govern: govern меняет то, что агент делает, так что его включение примешало бы крупный эффект к измерению меньшего.

2.3 Проверяемые предположения

Сформулированы как проверяемые утверждения, а не как критика:

A1 (доступность схемы)
Адекватная доменная схема существует до выполнения. Оригинал не даёт метода проектирования схемы и приводит дословный список полей лишь для одного из четырёх своих доменов[orig].

A2 (положительный эффект с первого хода)
Поддержание состояния в худшем случае нейтрально с первого хода. Кратчайший приводимый горизонт — $T{=}10$, что уже длиннее значительной доли реальных программных задач.

A3 (достаточная статистика)
Ничто отброшенное никогда больше не понадобится. Оригинал называет это ограничением[orig], но не измеряет его. Именно это предположение фальсифицирует данная статья.

M (валюта измерения)
Совокупное число токенов — валидный показатель для стоимости. Пол-

нотекстовый поиск по оригиналу даёт ноль вхождений слов cache, billing или иной денежной терминологии[orig].

A1 и A3 — свойства метода. M — свойство того, как метод оценивался, и именно оно затрудняет видимость двух других: метрика, считающая логические токены, не может отразить ни скидку кэша, которая вознаграждает дополняемый префикс, ни вызовы инструментов, потраченные на повторное приобретение того, что промпт больше не несёт.

2.4 Ось дизайна, используемая в этой статье

Вместо сравнения именованных архитектур мы фиксируем механизм и варьируем одно целое число: количество k самых последних дословных обменов, сохраняемых рядом с блоком состояния.

$$\text{промпт}_t \;=\; [\text{спецификация задачи}] \;+\; [\text{последние } k \text{ обменов}] \;+\; [\Sigma_t]$$

При $k{=}0$ это в точности skill.state в опубликованном виде. По мере роста k конфигурация интерполирует в сторону агента с транскриптом. Контрольная группа не является точкой на этой оси (у неё вообще нет блока состояния) и всегда приводится отдельно.

Это даёт три вещи, которые не может дать трёхрукое сравнение. Оно устраняет произвольную константу, которую незаметно внесла бы промежуточная «гибридная» рука; оно превращает вопрос из «какая архитектура» в «где на оси лежит оптимум»; и оно делает форму зависимости наблюдаемой, что в итоге и локализует ущерб (§7).

3 Формальная модель

Этот раздел фиксирует обозначения, формулирует два рантайма как алгоритмы и выводит тождество учёта, с которым сверяется остальная часть статьи. Асимптотический аргумент принадлежит оригиналу[orig]; наш вклад здесь — показать, какой из его членов стандартная оценка удерживает постоянным, и чем становится тождество, когда этот член освобождается.

3.1 Обозначения

Агент выполняет задачу за дискретные ходы $t = 1, 2, \ldots$ Пусть P — спецификация задачи, o_t — наблюдение, возвращённое средой на ходу t, a_t — действие, которое совершает агент, а R_t — его промежуточное рассуждение. Запишем Σ_t для структурированного состояния выполнения и C_t для контекста, собранного для обращения к модели на ходу t. Мы используем $|\cdot|$ для размера в токенах.

Горизонт T — это число ходов, которое эпизод фактически занимает. Является ли T входом или выходом — различие, на котором держится §6, и мы отмечаем его явно: T^{env}, когда его фиксирует среда, T^{ag}, когда агент останавливается по собственной воле.

3.2 Две политики контекста

Рантайм с транскриптом дополняет каждый обмен, так что его контекст на ходу t равен

$$C_t^{\text{tr}} \;=\; \big[\, P,\; (o_1, R_1, a_1), \ldots, (o_{t-1}, R_{t-1}, a_{t-1}),\; o_t \,\big], \tag{1}$$

что растёт линейно по t, давая $|C_t^{\mathrm{tr}}| = \mathcal{O}(t)$ и совокупный объём промпта

$$\sum_{t=1}^{T} \bigl|C_t^{\mathrm{tr}}\bigr| \;=\; \mathcal{O}(T^2). \tag{2}$$

Рантайм со структурированным состоянием отбрасывает историю и несёт только состояние:

$$C_t^{\mathrm{st}} \;=\; \bigl[\, P,\; \Sigma_{t-1},\; o_t \,\bigr], \qquad \bigl|C_t^{\mathrm{st}}\bigr| \;=\; \mathcal{O}\bigl(|P| + |\Sigma| + |o|\bigr), \tag{3}$$

что не зависит от t, если состояние ограничено, так что

$$\sum_{t=1}^{T} \bigl|C_t^{\mathrm{st}}\bigr| \;=\; \mathcal{O}(T). \tag{4}$$

Уравнения (2) и (4) — это опубликованный аргумент, и мы воспроизводим его без возражений[orig].

3.3 Семейство хвостов, интерполирующее между ними

Мы обобщаем оба в одно семейство, индексированное целым числом $k \geq 0$ — числом самых последних обменов, сохраняемых дословно рядом с состоянием:

$$C_t^{(k)} \;=\; \bigl[\, P,\; (o_{t-k}, R_{t-k}, a_{t-k}), \ldots, (o_{t-1}, R_{t-1}, a_{t-1}),\; \Sigma_{t-1},\; o_t \,\bigr]. \tag{5}$$

При $k = 0$ это в точности (3); при $k \to t$ оно восстанавливает (1) с избыточным блоком состояния. Контрольная группа вообще не несёт блока состояния и поэтому не является членом семейства; она приводится отдельно на протяжении всей статьи.

Каждое условие в этой статье — значение k в (5), что и превращает сравнение в позицию на кривой, а не в состязание между именованными системами.

3.4 Рантаймы как алгоритмы

Algorithm 1 Рантайм с транскриптом (контрольная группа)

Require: спецификация задачи P

1: $H \leftarrow \varnothing$ ▷ история взаимодействия
2: **for** $t = 1, 2, \ldots$ **do**
3: получить наблюдение o_t
4: $C_t \leftarrow [\,P,\ H,\ o_t\,]$
5: сгенерировать $(R_t, a_t) \sim \pi(C_t)$
6: выполнить a_t
7: $H \leftarrow H \,\|\, (o_t, R_t, a_t)$
8: **if** агент объявляет завершение **then**
9: **break**
10: **end if**
11: **end for**

Algorithm 2 Рантайм состояния с хвостом k

Require: спецификация P; схема S; хвост k

1: $\Sigma_0 \leftarrow \varnothing$; $H \leftarrow \varnothing$
2: **for** $t = 1, 2, \ldots$ **do**
3: получить наблюдение o_t
4: $C_t \leftarrow [\,P,\ \mathrm{tail}_k(H),\ \Sigma_{t-1},\ o_t\,]$
5: сгенерировать $(R_t, \Delta\Sigma_t, a_t) \sim \pi(C_t)$
6: **if** $\mathrm{valid}_S(\Delta\Sigma_t)$ **then**
7: $\Sigma_t \leftarrow \Sigma_{t-1} \oplus \Delta\Sigma_t$
8: **else**
9: $\Sigma_t \leftarrow \Sigma_{t-1}$; вернуть ошибку модели
10: **end if**
11: выполнить a_t; отбросить R_t
12: $H \leftarrow H \,\|\, (o_t, R_t, a_t)$
13: **if** агент объявляет завершение **then**
14: **break**
15: **end if**
16: **end for**

Два рантайма различаются ровно в одном: что собирает строка 4. Слияние $\oplus$ применяет проверенный по схеме патч, а недействительный патч оставляет состояние нетронутым. При $k{=}0$ хвост пуст, и Алгоритм 2 — это skill.state в опубликованном виде. Оба отбрасывают рассуждение R_t; только Алгоритм 2 также удаляет историю из контекста.

Проверка завершения в обоих (строка 8 Алгоритма 1 и строка 13 Алгоритма 2) — это собственное объявление агентом завершения, а не истечение внешнего счётчика. Это единственное структурное отличие от формулировки оригинала, где цикл выполняется для фиксированного T^{env}, и §6 показывает, что именно это отличие имеет значение.

3.5 Что упускает стандартный учёт

Пусть $\mathcal{I}_t$ обозначает информацию, которая нужна агенту на ходу t для выбора правильного действия. Она может прийти по двум путям: быть внесённой в контекст или быть заново получена из среды за счёт хода, потраченного на действие осмотра. Запишем ρ для доли ходов, потраченных на повторное приобретение.

Агент с транскриптом уже заплатил за свою историю (она в промпте), так что ρ^{tr} мало. Агент состояния отбросил всё, что не кодирует Σ, так что любой необходимый факт вне Σ должен быть заново получен, и $\rho^{(k)}$ растёт по мере уменьшения k. Поскольку каждое повторное приобретение потребляет ход, реализованный горизонт раздувается:

$$T^{\mathrm{ag}}(k) \;\approx\; \frac{T_0}{1 - \rho(k)}, \tag{6}$$

где T_0 — число ходов, которое потребовалось бы агенту с идеальной памятью.

Подставляя (6) в (4), получаем величину, которую асимптотика оригинала не содержит:

$$\sum_{t=1}^{T^{\mathrm{ag}}} \bigl|C_t^{\mathrm{st}}\bigr| \;=\; \mathcal{O}\!\left(\frac{T_0}{1-\rho}\right), \qquad \rho = \rho(k). \tag{7}$$

Опубликованное сравнение оценивает (4) против (2) при $T = T^{\mathrm{env}}$, зафиксированном равным для всех рук. Этот выбор устанавливает ρ вне модели: при числе ходов, зафиксированном средой, повторное приобретение не может удлинить эпизод, и (6) вырождается в $T^{\mathrm{ag}} = T^{\mathrm{env}}$ для каждого рантайма независимо от того, сколько было забыто.

Это центральный формальный тезис статьи: асимптотическое преимущество (4) держится по переменной (T), которую оценка удерживает постоянной, тогда как проверяемый механизм (ρ) действует именно на эту переменную. Мы измеряем ρ напрямую в §8 и T^{ag} в §7.

3.6 Стоимость — это не число токенов

Пусть α, γ, β — цены за токен для некэшированного входа, кэшированного входа и выхода соответственно. При h_t — кэшированной доле промпта на ходу t — тарифицируемая стоимость эпизода равна

$$\text{Стоимость} \;=\; \sum_{t=1}^{T} \Bigl[\, \alpha\,(1-h_t)\,|C_t| \;+\; \gamma\, h_t\,|C_t| \;+\; \beta\,|y_t| \,\Bigr], \tag{8}$$

где y_t — сгенерированный вывод, который для рантайма состояния включает патч $\Delta\Sigma_t$. У современных провайдеров $\gamma \ll \alpha < \beta$.

Отсюда следуют два следствия, оба измеряются в §9. Контекст транскрипта только дополняется, так что C_t^{tr} продолжает C_{t-1}^{tr} и $h_t \to 1$: растущий промпт из (2) тарифицируется в основном по дешёвой ставке γ. Блок состояния переписывается на каждом ходу, так что префикс расходится начиная с Σ, и h_t остаётся низким несмотря на маленький $|C_t|$. Одновременно поддержание состояния переносит токены из членов $|C_t|$ в $|y_t|$, что несёт самую высокую цену β.

Отсюда тождество, на котором построена эта работа: минимизация $\sum_t |C_t|$ и минимизация (8) — это разные цели, и на измеряемом здесь корпусе у них противоположные решения.

4 Дизайн исследования и заморозка протокола

Этот раздел намеренно помещён перед результатами. Дизайн был зафиксирован письменно до начала серии, и последовательность гипотеза → замороженный метод → данные → интерпретация — это то, что позволяет читателю отличить находку от истории, собранной постфактум.

4.1 Что было заморожено и когда

Предрегистрация — это артефакт репозитория, датированный раньше первого запуска серии[code]. Она фиксирует вопрос, гипотезы с условиями их фальсификации, вмешательство, зависимые переменные, вмешивающиеся факторы и контроль над ними, объём выборки с обоснованием мощности, правило

остановки, статистические тесты и политику исключений. Её раздел поправок пуст: ничего не менялось после начала первого запуска.

Три обязательства в ней делают настоящую работу:

1. **Вердикт внешний.** Исход запуска — это код завершения собственного верификатора задачи, никогда не отчёт агента о собственном успехе. Запуск без вердикта верификатора вообще не попадает в набор данных.
2. **Тесты выбраны до данных.** Точный парный тест Макнемара для способности решать задачи, точный знаковый тест на попарных отношениях для стоимости, ранговая корреляция Спирмена для тренда по горизонту. Ни один тест не выбирался после того, как был замечен эффект.
3. **Исключения определены до того, как были увидены подсчёты.** Правило контаминации (§12) и правило схлопывания дубликатов были оба написаны до их применения. Второе сохраняет самый ранний запуск повторно запущенной ячейки, так что повтор не может продвинуть себя в выборку, выиграв.

4.2 Гипотезы и как каждая могла бы быть опровергнута

H1 (стоимость).
Медианное отношение стоимости только-состояния к контрольной группе превышает 1.0. Опровергнута, если отношение ≤ 1.0, знаковый тест не достигает значимости, или направление непоследовательно между тремя испытаниями.

H2 (горизонт).
Отношение стоимости падает с ростом горизонта, пересекая 1.0: накладные расходы амортизируются на длинных задачах. Опровергнута, если корреляция отсутствует, положительна, или никогда не приближается к 1.0 в наблюдаемом диапазоне.

H3 (способность).
Приводится с доверительным интервалом и трактуется как недостаточно мощная, если наблюдаемое несовпадение не превышает предвычисленное требование в шесть односторонних несовпадающих пар.

H2 — это гипотеза, которую данное исследование опровергает предрегистрированным образом, и мы сообщаем об этом в §9, а не откладываем её в сторону.

4.3 Множественные сравнения

Три гипотезы были предрегистрированы, и именно эти три несут подтверждающий вес исследования. Всё остальное, для чего эта статья приводит p-значение — разбивки по стратам, ступени лестницы §10, сравнения стоимости по условиям — вторично и разведочно, и к этому не применяется поправка на семейственность сравнений.

Мы формулируем это явно, потому что §B критикует оригинал за то, что он этого не делает, и тот же стандарт должен применяться здесь. Следствие конкретно: наш ключевой результат переживает любую поправку, которую можно было бы применить ($p = 2.1 \times 10^{-13}$ по основному сравнению против порога Бонферрони $0.05/3$ для подтверждающего семейства), но несколько вторичных результатов лежат достаточно близко к 0.05, чтобы её не пережить. Там, где

вторичный результат несёт вес для утверждения этой статьи, мы говорим об этом прямо и трактуем его как предварительный, а не установленный.

4.4 Заранее объявленная ловушка артефакта

Одна ловушка была названа заранее, потому что она дала бы приукрашенный результат: более низкая стоимость на длинных задачах в руке только-состояния могла бы отражать более частое исчерпание бюджета ходов, что является цензурированием, а не амортизацией. Предрегистрация поэтому исключает запуски, остановленные по бюджету, из агрегации стоимости и приводит их число по каждой страте отдельно. Ловушка оказалась реальной: распределение причин остановки в Таблице 4 показывает остановки по бюджету, сконцентрированные именно в этой руке.

4.5 Охват

Эта серия обращается к вопросам стоимости и горизонта и приводит способность решать задачи. Чувствительность к схеме и композиция состояния — отдельные дизайны с собственными предрегистрациями; их статус приведён в §10. [**PENDING:** если владелец решит опубликовать только E1, этот абзац станет опережающей ссылкой на вторую статью]

5 Экспериментальная установка

5.1 Агент, корпус и условия

Агент — немодифицированный кодирующий агент с открытым исходным кодом; механизм состояния — это плагин, заменяющий контекст, собираемый для каждого обращения к модели, и добавляющий проверенный инструмент патчей[code]. Версии, хэш содержимого плагина и хэш схемы зафиксированы для серии и записаны в каждой строке запуска (плагин `be0f5b73e526ba4a`, схема `7d8705aae4e124f1`)[live].

Корпус — Terminal-Bench 2.0. Из 89 задач, в принципе выполнимых, этот стенд достигает 87: две отказывают при настройке контейнера, до того как агент начинает работу, при каждой попытке и в каждом условии[live]. Модель — `zai/glm-5.3-flash` на протяжении всего исследования.

Четыре условия различаются ровно в одном свойстве:

Условие	Контекст, собираемый для каждого обращения к модели
control	полный транскрипт, без блока состояния (немодифицированный агент)
state-only	блок состояния заменяет транскрипт ($k{=}0$)
state$+2$	блок состояния + 2 самых последних обмена
state$+6$	блок состояния + 6 самых последних обменов

Всё остальное заморожено: та же модель, схема, инструменты, бюджет ходов, задача и верификатор. Реализованное значение k считывается обратно из трассы каждого запуска, и запуск, чьё реализованное значение расходится с запрошенным, отклоняется, а не анализируется[code].

5.2 Единица, парность и повторение

Экспериментальная единица — один запуск одной задачи при одном условии в одном испытании. Каждое сравнение парное внутри (задачи, испытания) против контрольной группы того же испытания, никогда против объединённой контрольной группы: единичные запуски идентичной ячейки были измерены на 15, 51 и 65 обращениях к модели[live], так что объединение между испытаниями сравнивало бы условие с другой выборкой того же шума. Три независимых испытания дают 1044 запусков в сумме.

5.3 Вердикты

Вердикт — это код завершения собственного верификатора задачи. Это не формальность: на более ранней фазе этого проекта самоотчёт модели и сломанный контейнер верификатора вместе произвели целую серию строк, которые выглядели как данные, но ими не были (§12).

Запуски дополнительно классифицируются по тому, как они завершились (output_ceiling, solved, turn_budget, unsolved), и распределение приводится в Таблице 4, потому что оценка остановки по бюджету или остановки по потолку вывода как обычного провала приписывает методу свойство харнеса.

5.4 Учёт стоимости

Использование по вызовам (входные, кэшированные для чтения и выходные токены) берётся из журнала сессии. Стоимость выводится из этих измеренных токенов по одной фиксированной опубликованной таблице цен, а не читается у провайдера, по двум измеренным причинам: два кошелька аккаунта тарифицируют идентичное число токенов ровно в два раза по-разному[live], а путь через кастомный эндпоинт сообщает нулевую стоимость, возвращая при этом реальные токены[live]. Собственная цифра провайдера сохраняется в сырых записях для аудита.

Кэшированные для чтения токены отслеживаются отдельно от входных токенов на протяжении всего исследования, потому что центральное методологическое утверждение состоит в том, что это разные валюты. Мы проверили перед серией, что кэш префикса провайдера действительно видим в полях использования: повторный вызов сдвинулся от 0 до 3008 кэшированных токенов из 3019[live].

5.5 Проверки, выполняемые перед любым измерением

Стек, который был изменён, проходит дымовое тестирование, прежде чем ему доверяют[code]:

1. эталонное решение проходит верификатор, устанавливая, что верификатор вообще способен распознать правильный ответ;
2. каждая рука демонстративно загружает плагин и реализует заявленное значение k, проверяемое по собранному промпту, а не по собственному логу плагина; молчаливый сбой импорта иначе превращает экспериментальную руку в контрольную группу под другим именем, а числа всё ещё выглядят правдоподобно;
3. версии и хэши содержимого зафиксированы и записаны для каждого запуска.

Проверка 2 поймала реальный дефект стейджинга в этом проекте[live], поэтому она проверяет промпт, обращённый к провайдеру, а не внутреннюю трассу.

6 Воспроизведение: что исходный дизайн может и не может выразить

Прежде чем измерять расхождение, стоит установить, что расхождения с собственными числами оригинала нет.

6.1 Опубликованное преимущество переживает учёт с учётом кэша

Мы пересчитываем итоговую таблицу оригинала по нашей модели стоимости: их приведённые числа токенов, наши измеренные тарифы за токен и наши измеренные показатели попадания в кэш для двух архетипичных рантаймов. Результат сужает их преимущество, не переворачивая его[live]:

Горизонт T	отношение токенов	отношение стоимости при учёте кэша
10	1.6×	**0.97**×
25	2.9×	1.75×
50	5.7×	3.42×
100	19.0×	11.45×
200	21.3×	12.82×

Два прочтения. Кэширование примерно вдвое сокращает преимущество, но не инвертирует его: внутри исходного дизайна метод остаётся выигрышным по стоимости, и мы говорим об этом. А при $T{=}10$ отношение уже пересекло 1.0: паритет по стоимости исчезает на самом коротком горизонте их собственной таблицы, незамеченный, потому что их метрика не может это увидеть. Это A2, фальсифицированное на опубликованных данных, без новых запусков.

Эта оценка переносит наши тарифы на их токены и помечена как таковая: порядок величины сохраняется при их приведённых числах, но точные множители не являются измерениями их расходов.

6.2 Свойство, скрывающее повторное приобретение

В исходном генераторе горизонт T — это подсчёт событий среды, и каждый рантайм отвечает на каждое событие ровно одним обращением к модели[orig]. Число обращений к модели поэтому зафиксировано по построению и одинаково во всех руках.

Именно это единственное свойство определяет, что оценка может наблюдать. Доминирующая стоимость отбрасывания истории — это повторное приобретение отброшенного, а повторное приобретение тратится на дополнительные шаги, инициированные агентом. В дизайне, где число шагов — это вход, этой стоимости негде проявиться. Агент, забывший факт и вынужденный заново его вывести, всё равно получает ровно T обращений; забывание невидимо в журнале, который ведёт статья.

Таблица 1: Доля решённых задач по условиям. 1044 запусков, 87 задач × 3 испытаний. Интервалы — по Уилсону.

Condition	Solved / runs	Rate	95% CI
Control (full transcript)	160 / 261	0.613	[0.553, 0.670]
State-only (k=0)	91 / 261	0.349	[0.293, 0.408]
State +2	118 / 261	0.452	[0.393, 0.513]
State +6	154 / 261	0.590	[0.529, 0.648]

В агентной постановке число шагов — это результат. На нашем корпусе рука только-состояния занимает 2.22× от числа ходов контрольной группы на задачах, которые обе руки решили[live], и это отношение измерено только там, где метод преуспел, так что это нижняя граница накладных расходов, а не среднее по его провалам.

6.3 Что это подтверждает и что нет

Механика оригинала воспроизводится, его утверждение о токенах воспроизводится, и его знак переживает кэширование внутри его собственного дизайна. Не переносится вывод из этого дизайна на агентное выполнение. Обе оценки легитимны; они отвечают на разные вопросы, и только один из них — тот вопрос, который практик задаёт перед тем, как встроить архитектуру памяти в кодирующего агента.

Одна оговорка о точности воспроизведения, приведённая здесь, а не спрятанная: наш блок состояния толще на шаг, чем приведённая цифра оригинала[live], так что наши абсолютные величины стоимости — это верхние границы. Более тонкий блок усилил бы сокращение размера промпта и ослабил бы инверсию стоимости; оба движутся против нашего ключевого результата, поэтому мы приводим оба. [**NOTE:** рука верности исходной статье, которая изолирует это, построена и предрегистрирована; если она запустится до подачи статьи, её числа заменят этот абзац]

7 Результаты I: способность решать задачи

Рисунок 2 обобщает этот раздел и два следующих; таблицы ниже приводят те же величины в точности.

7.1 Доля решённых задач

Таблица 1 даёт непарную картину. Опубликованная конфигурация решает 0.349 запусков против 0.613 у контрольной группы, и интервалы не пересекаются. Восстановление истории монотонно восстанавливает долю: 0.452 при двух обменах, 0.590 при шести.

7.2 Парное сравнение

Непарные доли недооценивают дизайн: каждое условие проходило каждую задачу, так что информативная величина — это множество задач, по которым две руки расходятся. Таблица 2 сравнивает каждую руку с контрольной группой того же испытания.

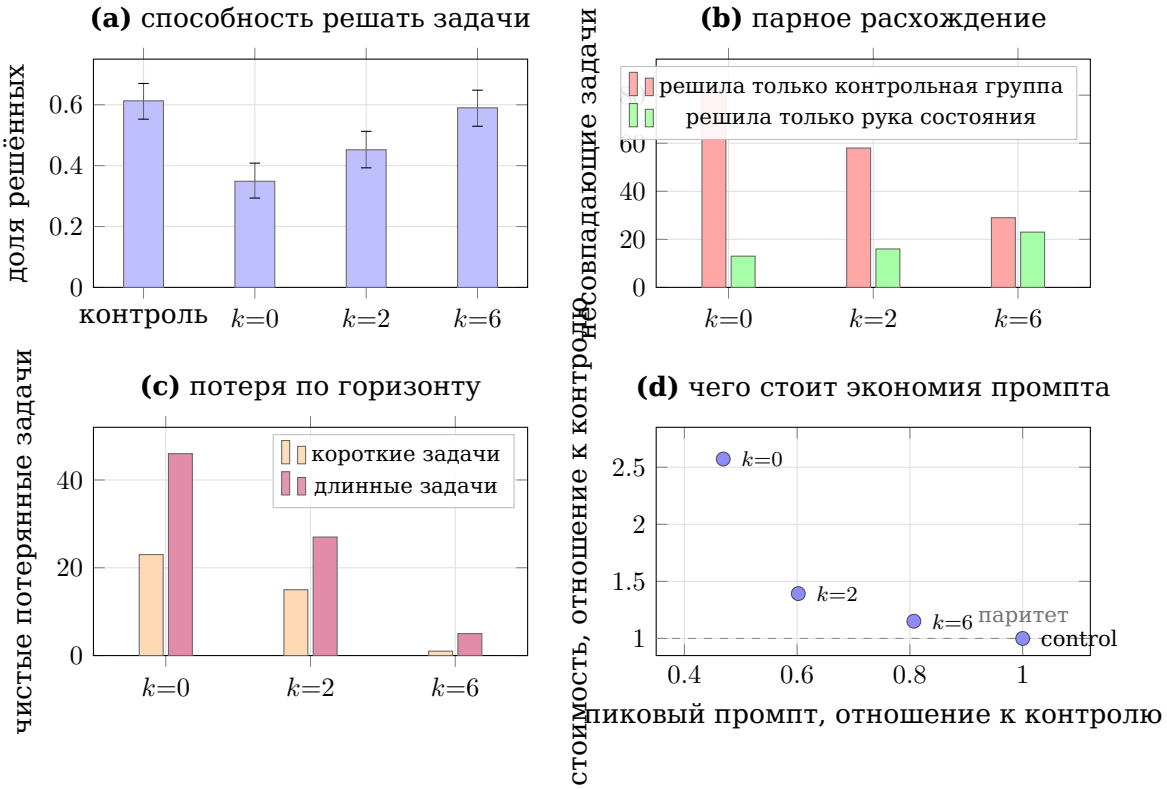


Рис. 2: Характеристика памяти со структурированным состоянием на 87 программных задачах, 1044 запусков. **(a)** Доля решённых задач с интервалами Уилсона; контрольная группа решает 0.613 против 0.349 для опубликованной конфигурации. **(b)** Парное расхождение против контрольной группы того же испытания — только несовпадающие задачи несут информацию, и они подавляюще односторонни при $k{=}0$. **(c)** Чистые потерянные задачи, разбитые по медианному горизонту контрольной группы: дефицит расширяется на длинных задачах, инвертируя аргумент в пользу метода. **(d)** При равном исходе опубликованная конфигурация вдвое сокращает пиковый промпт и умножает стоимость на 2.57. Восстановление шести обменов ($k{=}6$) устраняет дефицит способности решать задачи и сокращает премию по стоимости с 2.57× до 1.15×, но не опускается ниже контрольной группы: при равном исходе она остаётся более дорогой рукой ($p = 0.04$).

Опубликованная конфигурация проигрывает 82 задач и выигрывает 13 ($p = 2.1 \times 10^{-13}$). Эффект велик, односторонен и далеко за порогом значимости. При двух сохранённых обменах дефицит всё ещё ясен (58 против 16, $p = 9.7 \times 10^{-7}$). При шести он исчезает: 29 против 23 неотличимо от шума ($p = 0.49$).

Эта последняя строка — самая информативная в статье. Шесть дословных обменов — небольшое добавление к промпту, однако они восстанавливают способность решать задачи до паритета. Значит, ущерб наносит не суммирование состояния и не его структурирование: это отбрасывание недавнего транскрипта. То, что блок состояния стоит в способности решать задачи, возмещается историей, которую он заменяет, а не самим блоком состояния.

7.3 Горизонт: потеря растёт там, где была обещана выгода

Задачи разбиты по медианному числу ходов контрольной группы (11 ходов), давая 45 коротких и 42 длинных задач. Горизонт берётся из контрольной руки: использование собственного числа ходов экспериментальной руки было

Таблица 2: Парная способность решать задачи против контрольной группы того же испытания, точный тест Макнемара. Информацию несут только несовпадающие ячейки.

Condition	Pairs	Control only	Arm only	Both	p
State-only ($k{=}0$)	261	82	13	78	2.1×10^{-13}
State $+2$	261	58	16	102	9.7×10^{-7}
State $+6$	261	29	23	131	0.49

Таблица 3: Парная способность решать задачи по страте горизонта, разбитой по медиане контрольной группы в 11 ходов.

Condition	Horizon	Control only	Arm only	p
State-only ($k{=}0$)	short	28	5	6.6×10^{-5}
State-only ($k{=}0$)	long	54	8	1.7×10^{-9}
State $+2$	short	23	8	0.01
State $+2$	long	35	8	4.2×10^{-5}
State $+6$	short	11	10	1.00
State $+6$	long	18	13	0.47

бы циклическим, поскольку рука, заново получающая данные, по построению занимает больше ходов.

Для опубликованной конфигурации дефицит составляет 28 против 5 на коротких задачах ($p = 6.6 \times 10^{-5}$) и расширяется до 54 против 8 на длинных ($p = 1.7 \times 10^{-9}$).

Это инвертирует аргумент в пользу метода. Структурированное состояние мотивируется длинными горизонтами, где неограниченный транскрипт предположительно становится неуправляемым. На этом корпусе именно длинный горизонт — там, где отбрасывание транскрипта стоит дороже всего, так что дефицит накапливается в течение задачи в том же направлении, в котором предполагалась выгода.

При $k{=}6$ ни одна страта не показывает различия, так что эффект горизонта — это свойство того, сколько истории было удалено, а не свойство блока состояния.

7.4 Как завершались запуски

Сравнение способности решать задачи честно только тогда, когда ограничение харнеса не засчитывается как неспособность. Таблица 4 разделяет четыре терминальных состояния.

Исчерпание бюджета ходов появляется в руках, несущих состояние, и никогда — в контрольной группе или при $k{=}6^{\mathsf{live}}$. Это само по себе свидетельство в пользу механизма из §8: рука, вынужденная заново получать свой контекст, — это рука, у которой заканчиваются ходы. Это также заранее объявленная ловушка артефакта из §4, и именно поэтому каждое утверждение о стоимости в §9 вычисляется только по парам, где обе руки действительно решили задачу.

Таблица 4: Причины остановки по условиям. Остановки по бюджету сконцентрированы в руке, занимающей больше всего ходов, поэтому они исключены из агрегации стоимости.

Condition	output_ceiling	solved	turn_budget	unsolved
Control (full transcript)	62	160	0	39
State-only (k=0)	117	91	34	19
State +2	95	118	18	30
State +6	67	154	0	40

8 Результаты II: механизм

Раздел 7 устанавливает, что отбрасывание недавней истории стоит способности решать задачи. Этот раздел устанавливает, как эта плата вносится, на основе вызовов инструментов тех же запусков.

8.1 Агент выкупает обратно то, что отбросил промпт

Запуск только-состояния не проваливается из-за забывания плана. Он проваливается из-за повторного вывода того, что уже было пронаблюдано. Классификация каждой команды оболочки из 472 запусков с полными трассами по назначению и подсчёт команд, повторяющих ранее идентичный вызов[live]:

Условие	Команд / запуск	Доля осмотра	**Повторный осмотр**
control (полная история)	95	0.23	**0.20**
state-only, решено	163	0.52	0.45
state-only, провалено	278	0.67	**0.58**

Неуспешный запуск только-состояния тратит 58 % своих команд на повторное выполнение уже выполненного осмотра, против 20 % у контрольной группы. На отдельной матрице тот же эффект проявляется как частота перехода по указателям в 5.19 на запуск против 0.42 у контрольной группы, примерно в двенадцать раз чаще[live].

Блок состояния ограничивает промпт без ограничения информации, требуемой задачей, и недостача покрывается через цикл инструментов, куда не заглядывает ни одна метрика размера промпта. Это тезис данной статьи, сформулированный как измерение, а не как аргумент.

8.2 Прочитано вручную

Одна траектория, изученная напрямую, а не через агрегат[live]: 72 вызова инструмента, из которых `input.tex` прочитан 22 раза при двух написаниях его пути, второй файл — 19 раз, а документ перекомпилирован пять раз, возвращая те же семь предупреждений на тех же номерах строк при каждом вызове. Ходы 22–33 почти целиком состоят из чередующихся повторных чтений двух файлов, байты которых ни разу не менялись.

Агрегаты может производить и сломанная метрика; этот же был сначала прочитан вручную, а метрика написана позже, чтобы механически его воспроизвести.

8.3 Что фактически содержало состояние

Изучение финального состояния 251 запуска только-состояния по исходу[live]:

Маркер, присутствующий в финальном состоянии	Решено	Провалено
Символов (медиана)	1218	963
Пути к файлам	9	5
Точные строки версий	1	**0**
Текст ошибки или кода выхода	0	**0**
Процитированные команды	1	**0**

Состояние неуспешного запуска несёт ноль точных версий, ноль текста ошибок и ноль команд — именно те факты, которые он затем заново выводит, повторно запуская ту же команду. Схеме некуда их поместить: её поля — это текстовые ёмкости прозы (`goal`, `facts_established`, `work_done`, `work_remaining`, `next_step`, `current_blocker`).

Связь проявляется в ожидаемую сторону[live]: запуски, чьё состояние фиксирует не менее восьми путей к файлам, решаются с частотой 0.50 против 0.24 для тех, что фиксируют меньше; запуски, фиксирующие точную версию, решаются с частотой 0.47 против 0.32; запуски, фиксирующие ошибку или код выхода, — с частотой 0.50 против 0.33. Это корреляционно (запуск, продвинувшийся дальше, имел больше материала для записи), но это единственное направление, на которое указывают данные, и оно мотивирует исследование композиции в §10.

8.4 Почему это не может проявиться в исходной оценке

Повторное приобретение тратится в виде дополнительных шагов, инициированных агентом. Генератор оригинала фиксирует число шагов на эпизод (§6), так что механизм, измеряемый в этом разделе, не имеет канала, через который мог бы проявиться. Это конкретный смысл, в котором синтетический результат не ошибочен, а неприменим: бенчмарк устраняет переменную, которая несёт стоимость.

9 Результаты III: экономика

9.1 Стоимость при равном исходе

Сравнение стоимости между руками, решающими разное число задач, бессмысленно: неуспешный запуск останавливается раньше и потому стоит дешевле. Каждое отношение в Таблице 5 вычислено только по парам, где обе руки решили одну и ту же задачу в одном и том же испытании, и каждое отношение — это медиана по парам, так что одна дорогая задача не может задать число.

При равном исходе опубликованная конфигурация несёт 0.47× от пикового промпта контрольной группы и стоит 2.57× дороже ($p = 1.4 \times 10^{-12}$). Промпт вдвое меньше, а счёт в два с половиной раза больше.

Таблица 5: Отношения ресурсов против контрольной группы того же испытания, по парам, где обе руки решили задачу. Медианы по парам; p из точного знакового теста по отношениям стоимости.

Condition	Pairs	Cost	Peak prompt	Output	Cached	p
State-only (k=0)	78	2.57×	0.47×	4.55×	0.89×	1.4×10^{-12}
State +2	102	1.39×	0.60×	2.08×	0.40×	4.7×10^{-4}
State +6	131	1.15×	0.81×	1.36×	0.21×	0.04

Таблица 6: Итоги по серии. Приводятся только для масштаба: суммы по условию — нечестное сравнение, потому что неуспешный запуск останавливается раньше.

Condition	Cost	Turns	Input	Cached	Output
Control (full transcript)	$17.85	6,504	18.7M	352.2M	9.0M
State-only (k=0)	$14.83	16,072	27.3M	25.9M	19.9M
State +2	$12.88	10,250	27.6M	15.9M	16.5M
State +6	$11.83	4,725	38.9M	6.2M	11.6M

9.2 Куда уходят деньги

Та же таблица несёт механизм. Выходные токены составляют 4.55× от контрольной группы: блок состояния регенерируется на каждом ходу, а сгенерированный текст тарифицируется по самому дорогому тарифу. Кэшированные чтения убывают по мере сокращения сохраняемой истории, и итоги по серии показывают тот же эффект с другой стороны: контрольная группа обслуживается 352.2M кэшированных токенов по всей серии против 6.2M для state+6^{live}.

Только дополняемый транскрипт почти полностью обслуживается кэшем, потому что каждый вызов разделяет префикс предыдущего вызова. Блок состояния, переписываемый на каждом ходу, расходится с предыдущим префиксом в месте, где начинается блок, так что скидка, делающая контрольную группу дешёвой, не применяется. Ни одна из этих величин не видна в совокупном подсчёте токенов, что и составляет конкретное содержание методологического утверждения: сокращение токенов и сокращение стоимости — разные измерения, и на этом корпусе у них противоположные знаки.

9.3 Решённые задачи на единицу затрат на уровне серии

Рантайм, вдвое сокращающий число токенов и проваливающий половину задач, — не улучшение. Взгляд с точки зрения эффективности (Таблица 7) делит исходы на ресурсы.

Из этого следуют две вещи. На токен руки состояния действительно выглядят эффективными (1.24 против 0.42 решённых на миллион у контрольной группы); это метрика оригинала, и это прочтение, которое даёт оценка по числу токенов. На доллар ранжирование меняется: state-only — худшая из всех четырёх рук при 6.1 решённых на доллар, ниже контрольной группы с 9.0.

По этой мере на уровне серии лучшая рука — state+6 при 13.0 решённых на доллар против 9.0. Это сравнение нужно читать с оговоркой из §9.1: неуспешный запуск останавливается раньше и стоит немного, так что итоги по серии приукрашивают ту руку, которая проваливает больше, и они приводятся здесь для масштаба, а не как вердикт.

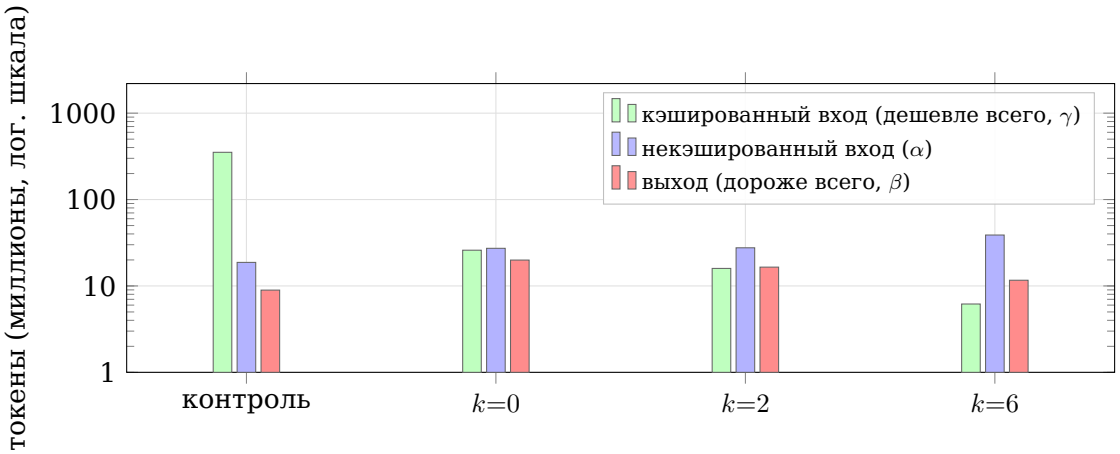


Рис. 3: Потребление токенов по тарифицируемым классам по всей серии, декомпозиция, которую тарифицирует Ур. (8). Столбцы сгруппированы, а не составлены друг на друга, а ось логарифмическая, потому что классы различаются на два порядка. Контрольная группа потребляет абсолютно больше всего токенов, но 352.2M из них — кэшированные чтения, тарифицируемые по самой дешёвой ставке: только дополняемый промпт почти полностью обслуживается из кэша префикса. Руки состояния потребляют намного меньше токенов и платят за них больше, потому что блок состояния, переписываемый на каждом ходу, теряет этот кэш (вплоть до 6.2M) и потому что поддержание состояния переносит токены в класс выхода. Именно поэтому сравнение по числу токенов и сравнение по стоимости расходятся.

Таблица 7: Эффективность. Решённых задач на миллион токенов и на доллар по всей серии.

Condition	Solved	Tokens	Cost	Solved/Mtok	Solved/$
Control (full transcript)	160	379.9M	$17.85	0.42	9.0
State-only (k=0)	91	73.1M	$14.83	1.24	6.1
State +2	118	60.1M	$12.88	1.96	9.2
State +6	154	56.7M	$11.83	2.72	13.0

Сравнение при совпадающем исходе — то, что решает дело. По нему state+6 не дешевле контрольной группы: 1.15× по 131 задачам, решённым обеими руками, дороже в 78 из них против 53 ($p = 0.04$)[live]. Честное утверждение экономики поэтому уже, чем предполагает подсчёт токенов в любом из направлений. Восстановление короткого хвоста устраняет дефицит способности решать задачи (§7) и снижает премию по стоимости с 2.57× до 1.15×, но на этом корпусе ни одна конфигурация структурированного состояния не оказалась дешевле простого сохранения транскрипта при равном исходе.

9.4 H2 опровергнута согласно предрегистрации

Предрегистрация предсказывала, что отношение стоимости упадёт с ростом горизонта и пересечёт 1.0, то есть что накладные расходы амортизируются на длинных задачах. Этого не происходит. Корреляция Спирмена между горизонтом контрольной группы и отношением стоимости по задаче составляет +0.16 по 35 задачам ($p = 0.36$) — корреляция, неотличимая от нуля и, если что, неверного знака.

Тест охватывает 35 из 87 задач по определению этой величины, а не по

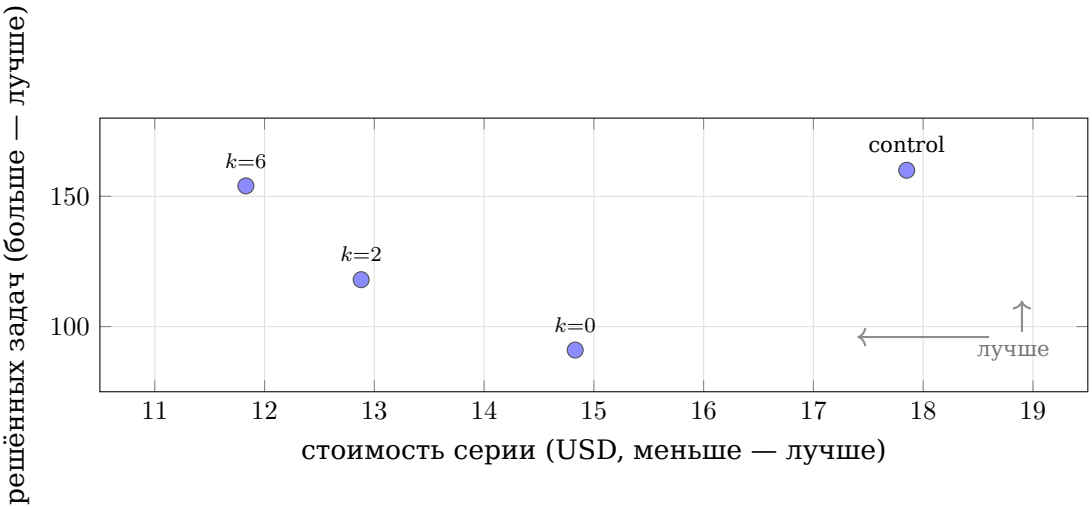


Рис. 4: Граница, которая имеет значение на практике: решённые задачи против потраченных денег, вся серия. Опубликованная конфигурация (k=0) доминируема: она тратит меньше контрольной группы, но решает намного меньше задач, и обе k=2 и k=6 превосходят её по обеим осям одновременно. Читайте этот рисунок только как обобщение на уровне серии. Поскольку неуспешный запуск останавливается раньше и потому стоит немного, итоги по серии приукрашивают руки, которые проваливаются чаще, поэтому именно сравнение при совпадающем исходе из Таблицы 5, а не этот рисунок, несёт утверждение статьи о стоимости. По этому честному сравнению k=6 не дешевле контрольной группы (1.15×, $p = 0.04$). Что рисунок действительно показывает — это то, что ни один из крайних случаев оси хвоста не является правильным ответом.

какому-либо правилу исключения: отношение стоимости по задаче существует только там, где и контрольная группа, и state-only решили задачу хотя бы в одном испытании, а state-only решает 0.349 запусков. Это ограничение следует из результата о способности решать задачи в §7, и оно играет против нас: задачи, где state-only провалилась полностью, — это как раз длинные задачи, на которых предсказывалась амортизация.

В наблюдаемом диапазоне горизонта пересечения нет. Мы сообщаем об этом как об опровержении собственной гипотезы, а не отбрасываем её, и это сходится с §7: не только стоимость не амортизируется, но и дефицит способности решать задачи расширяется с горизонтом.

10 Композиция состояния: кто пишет состояние

Разделы 7–9 устанавливают, что ущерб происходит от отброшенной истории и что агент платит за него, заново получая информацию через свои инструменты. Этот диагноз подразумевает исправление: если бы состояние уже несло факты, которые агент постоянно заново выводит, повторное приобретение не понадобилось бы.

Этот раздел варьирует одну вещь: сколько состояния пишет рантайм, не спрашивая модель. Это другая ось, отличная от хвоста, и она приводится отдельно, потому что отвечает на другой вопрос. Ось хвоста спрашивает, сколько истории хранить. Эта ось спрашивает, при фиксированном объёме истории (никакой), кто должен производить состояние, заменяющее её.



Рис. 5: Два способа ограничить промпт блоком состояния, различающихся тем, кто его пишет. **Слева:** модель выдаёт обновление состояния как сгенерированный текст, так что его поддержание тарифицируется по выходному тарифу, а модель вольна перезаписать ранее записанный факт, чему оригинал приписывает 68 % своих отказов моделей с открытыми весами[orig]. **Справа:** рантайм выводит ту же запись из результатов инструментов, которые уже произошли. Блок — это факт о том, что случилось, а не утверждение модели об этом, он не стоит сгенерированных токенов и не может быть перезаписан моделью. Полоса внизу — это фактор, варьируемый в §10: каждая ступень передаёт рантайму ещё один компонент, при замороженных хвосте и схеме, так что разрыв между соседями называет один делегированный компонент.

10.1 Фактор

В skill.state блок состояния — это сгенерированный текст: модель выдаёт $\Delta\Sigma_t$ в той же генерации, что и своё действие (Алгоритм 2). Из §9 следуют два следствия, оба неблагоприятны. Поддержание состояния тарифицируется по выходному тарифу β, самому дорогому классу в Ур. (8). И модель вольна перезаписать факт, который она записала раньше; собственная таксономия ошибок оригинала приписывает 68 % отказов моделей с открытыми весами именно этому[orig].

Альтернатива — позволить рантайму фиксировать то, что уже произошло. Результат работы инструмента — это пронаблюдённый факт, а не утверждение: рантайм видел, какие файлы были затронуты, какая команда выполнена, что она вывела и прошли ли тесты. Запись этого в состояние не требует генерации и не может быть перезаписана моделью.

Сколько записывать — это фактор. Лестница добавляет один компонент на ступень:

Таблица 8: Лестница композиции. Стоимость, ходы и вывод — медианы по задачам, которые решили и ступень, и контрольная группа; отношения ниже 1 благоприятствуют ступени.

Rung	Adds	Solved	Rate	Cost ratio	Turns	Output
control	full transcript	48	0.552	1.00×	1.00×	1.00×
v1	files	25	0.287	2.31×	1.62×	3.96×
v3	+ commands	35	0.402	1.20×	0.79×	1.64×
v6	+ tests	37	0.425	0.92×	0.71×	1.84×
v7	+ failures	36	0.414	0.94×	0.73×	1.73×

Ступень	Добавляет	Что пишет рантайм
v1	файлы	пути, к которым обращался агент
v2	+ содержимое	записи изменений по файлам и глобальные
v3	+ команды	нормализованные записи команд с их выводом
v4	+ диагностика	вывод компилятора и линтера, структурированный
v5	+ окружение	версии, пути, состояние интерпретатора
v6	+ тесты	вызовы тестов и их результаты
v7	+ провалы	записи о том, что пробовали и что не сработало

Хвост заморожен при $k{=}0$, а схема — на `L2-semantic` в каждой ячейке, так что ни одна ступень не заимствует преимущество у факторов из §7. Соседние ступени различаются ровно одним компонентом, и этот инвариант утверждается по флагам, которые условия фактически выдают, до запуска любой ячейки, а не по замыслу, стоящему за ними[code].

Контрольная группа выполняется внутри этой серии. Повторное использование контрольной группы E1 было опробовано, и данные его отвергли: та же рука на тех же 87 задачах набрала 0.793 в E1 и 0.552 здесь, разойдясь по 31 из них[live]. Стенд дрейфует между сериями, так что заимствованная контрольная группа измеряет дрейф наравне с условием. Против заимствованной контрольной группы первая ступень выглядела как дефицит 46–2; против контрольной группы внутри серии это 25–2. Одна контрольная группа на серию.

10.2 Способность решать задачи улучшается по мере делегирования

Таблица 8 приводит лестницу. Первая ступень — состояние, хранящее только список затронутых файлов, — решает 0.287 против 0.552 у контрольной группы. Добавление записей команд поднимает её до 0.402, тесты — до 0.425. Последняя ступень не продолжает подъём: добавление записей о провалах даёт 0.414— на одну задачу меньше, чем v6, что при 5 против 4 несовпадающих задач неотличимо от отсутствия изменения. Лестница поэтому не монотонна на своей вершине, хотя падение попадает в полосу шума, а не в порог парного сравнения.

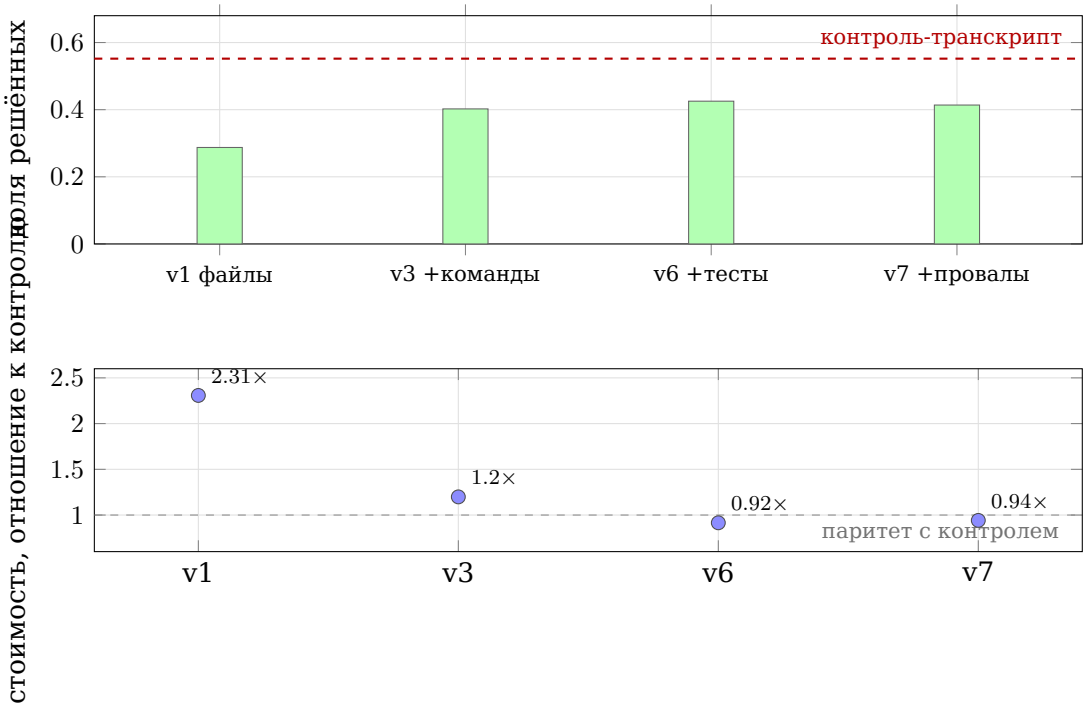


Рис. 6: Делегирование большей части состояния рантайму двигает способность решать задачи и стоимость в одном направлении, что противоположно обычному компромиссу. **Сверху:** доля решённых задач по ступени против контрольной группы транскрипта внутри серии (пунктир). Первая ступень восстанавливает 0.287 задач; передача команд, тестов и записей о провалах достигает 0.425, закрывая примерно половину дистанции до контрольной группы без восстановления какого-либо транскрипта. **Снизу:** медианная стоимость на задачах, которые решили и ступень, и контрольная группа. Те же добавления, что поднимают способность решать задачи, также снижают стоимость с 2.31× контрольной группы до 0.92×, ниже паритета. Состояние, выведенное рантаймом, дешевле состояния, написанного моделью, потому что оно никогда не генерируется по выходному тарифу.

Парные ступени приведены в Таблице 9. Значим сам по себе только первый шаг: переход от файлов к командам выигрывает 12 задач и проигрывает 2 ($p = 0.01$). Более поздние отдельные шаги при таком размере выборки неотличимы от шума по отдельности (9 против 11, $p = 0.82$; и 5 против 4, $p = 1.00$). Сквозной шаг значим: переход от v1 к v7 выигрывает 14 и проигрывает 3 ($p = 0.01$).

Мы поэтому утверждаем тренд, а не отдельные ступени. Данные подтверждают, что состояние, несущее больше выведенных рантаймом фактов, решает больше задач, чем состояние, несущее меньше, причём основная часть выигрыша приходит рано, в точке, где команды и их вывод входят в запись. Локализация оставшихся компонентов друг относительно друга требует испытаний, которые всё ещё выполняются.

10.3 Стоимость падает в то же время

Результат, который имеет значение, находится в последних трёх столбцах Таблицы 8, потому что способность решать задачи и стоимость движутся вместе, а не друг против друга.

На v1 рука стоит 2.31× от контрольной группы при равном исходе. Это режим §9: состояние, почти ничего не записывающее, вынуждает агента заново получать почти всё, и на это уходит 1.62× от числа ходов контрольной груп-

Таблица 9: Парные шаги вдоль лестницы, одна и та же задача и испытание.

Step	Lower solves only	Higher solves only	p
v1 → v3	2	12	0.01
v3 → v6	9	11	0.82
v6 → v7	5	4	1.00
v1 → v7	3	14	0.01

Таблица 10: Каждая ступень в паре с контрольной группой транскрипта внутри серии.

Rung	Control only	Rung only	p
v1	25	2	5.6×10^{-6}
v3	18	5	0.01
v6	17	6	0.03
v7	16	4	0.01

пы. К v6 то же сравнение составляет 0.92×, ниже контрольной группы, а ходы упали до 0.71×. Рука, делегирующая больше всего, одновременно наиболее способна на этой оси и наиболее дешёва.

Механизм виден в столбце вывода. На v1 рука генерирует 3.96× от выходных токенов контрольной группы; к v6 это 1.84×. За факты, которые пишет рантайм, не платят по выходному тарифу, так что перенос компонента от модели к рантайму убирает сгенерированные токены одновременно с устранением повторного приобретения, которое эти токены не могли предотвратить.

На доллар по всей серии порядок тот же: 12.9 решённых задач на доллар на v1 против 20.9 на v6, при контрольной группе на 22.4.

10.4 Чего это не показывает

Лестница не достигает контрольной группы. Каждая ступень остаётся позади неё по парной способности решать задачи (Таблица 10), и лучшая всё ещё имеет -11 задач в минусе против агента с транскриптом. Делегирование большего рантайму — строгое улучшение над состоянием, написанным моделью; на этом корпусе это ещё не замена сохранению недавней истории.

В сочетании с §7, где всего шесть дословных обменов полностью закрывают разрыв, практический вывод состоит в том, что два механизма дополняют, а не соревнуются друг с другом: состояние, написанное рантаймом, — дешёвый способ нести устойчивые факты, а короткий хвост — дешёвый способ нести недавний контекст. Ни один здесь не заменяет другой.

Статус этих чисел. 435 запусков, 87 задач, одно испытание. E1 несёт три испытания; эта ось несёт одно, и в этом проекте уже был эффект, значимый при двух испытаниях и умерший на третьем (§12). Направление согласуется на каждой ступени, и сквозное сравнение значимо, но отдельные шаги от ступени к ступени следует читать как предварительные до тех пор, пока не поступят оставшиеся испытания. Ни одно утверждение в этом разделе не опирается на собственные счётчики инструментирования плагина, которые

тождественно нулевые по всей серии; мы трактуем это как мёртвое инструментирование, а не как свидетельство отсутствия[live].

11 Угрозы валидности

Одно семейство моделей. Каждый запуск использует `zai/glm-5.3-flash`. Парный пилот на более сильной модели того же провайдера не решил ни одной задачи, которую не решила бы маленькая модель, при 22.7× медианной стоимости на ячейку[live], поэтому серия не была перенесена. Этот пилот — 7–8 задач, и он закрывает вопрос бюджета, а не вопрос переносимости. Мы ничего не утверждаем о моделях других семейств и, в частности, ничего о том, поддерживала бы состояние достаточно хорошо, чтобы избежать режима отказа из §8, модель с более сильным следованием инструкциям.

Один корпус и его диапазон горизонта. Задачи Terminal-Bench 2.0 используют медианно примерно 5 % окна контекста при контрольной группе[live]. Окно поэтому редко является связывающим ограничением, что как раз режим, в котором механизм памяти должен быть наименее необходим. Пилот на более длинном Terminal-Bench 4.0 показывает медианный пик в 66 % окна[live], и там тот же механизм проявляется на порядок сильнее (медианное повторение 3.26 против 1.02 у контрольной группы). Этот пилот — одно испытание на шести задачах, и это не результат. Честное утверждение охвата состоит в том, что наша находка держится там, где окно не насыщено, а направление эффекта в насыщенном режиме измерено, но не установлено.

Точность реализации. Наш блок состояния толще на шаг, чем сообщает оригинал[live], так что абсолютные величины стоимости — верхние границы (§6). Обе поправки движутся против нашего ключевого результата.

Перенос тарифов в воспроизведении. Раздел 6 применяет наши измеренные цены к опубликованным числам токенов оригинала. Порядок величины не зависит от точных множителей, которые представляют собой оценку и помечены как таковая.

Контаминация, в принципе не обнаруживаемая. Правило карантина (§12) опирается на ошибочные ходы. Деградация провайдера ниже порога видимости ошибок не была бы поймана. Граница ущерба состоит в том, что результаты по стоимости вообще не зависят от вердиктов, и что результат по способности решать задачи пережил удаление каждой ячейки, которую поймало правило.

Две невыполнимые задачи. Две из 89 задач отказывают при настройке контейнера при каждом условии[live] и отсутствуют во всех руках одинаково. Это убирает две задачи из корпуса; это не благоприятствует какому-либо условию.

Исследование композиции несёт одно испытание. Раздел 10 приводит 435 запусков за одно испытание, против трёх для E1. Учитывая собственную историю этого проекта (эффект, значимый при двух испытаниях, умерший на третьем (§12)), отдельные шаги от ступени к ступени там предварительны. Сквозное сравнение и направление тренда стоимости согласуются на каждой ступени, поэтому они вообще приводятся, но ни одно утверждение в аннотации или заключении этой статьи на них не опирается.

Нет руки только-хвоста. Четыре условия E1 — это контрольная группа без блока состояния и три руки, несущие состояние, с разными хвостами. Нет руки, несущей последние k обменов и не несущей блока состояния. Без неё §7 может показать, что восстановление истории устраняет дефицит, но не может отделить, какая часть оставшегося поведения при $k{=}6$ приходит от блока состояния, а какая от истории рядом с ним. Таблица 6 содержит намёк на то, что это важно: state$+6$ завершает серию за меньшее общее число ходов, чем контрольная группа, что чисто аддитивный учёт двух компонентов не предсказывает очевидным образом. Мы отмечаем это как необъяснённое, а не интерпретируем, и этой руке место в следующей серии.

12 Подводные камни измерения и два отзыва

Оценки памяти агентов имеют характерный режим отказа: числа реальны, скрипт верен, а метка на столбце ошибочна. Каждый пункт ниже стоил этому проекту серии, прежде чем стать правилом, и каждый приводится потому, что правило переносится даже там, где наши результаты — нет.

12.1 Пять подводных камней

1. **Самоотчёт модели — не вердикт.** Решает только собственный верификатор задачи. Каждая альтернатива, которую мы пробовали, производила строки, выглядевшие как данные, но ими не бывшие.
2. **Экспериментальная рука может незаметно стать контрольной группой.** Плагин, не загрузившийся, оставляет руку работающей как базовая линия под другим именем, а её числа остаются правдоподобными. Проверка утверждает реализованный состав контекста против промпта, обращённого к провайдеру, а не против собственного лога плагина[code].
3. **Детектор может совпадать с конфигурацией вместо событий.** Детектор тайм-аутов в этом проекте совпал со статической строкой политики повторов, печатаемой в каждом логе, сообщив о 54 тайм-аутах там, где их не было[live]. Дефект проявился лишь через невозможное нижестоящее число: долю решённых задач выше 100 %. Его модульный тест закодировал ту же ошибку, так что набор тестов был зелёным, потому что код и тест ошибались идентично. Принятое правило: детектор проверяется на запуске с известной заранее истиной, прежде чем ему разрешат приблизиться к корпусу.
4. **Насыщение провайдера читается как отказ метода.** При конкурентности выше измеренного потолка провайдера 68 % запросов были ограничены по скорости[live]. Изголодавшийся запуск всё равно получает честный ноль от своего верификатора, и этот ноль входит в таблицу как «ме-

тод провалился». Контаминация однонаправленна по построению: контрольная группа уже завершилась в чистых условиях, так что ходы потеряли только экспериментальные руки. Правило карантина было зафиксировано до того, как были увидены подсчёты: чистым считается ноль ошибочных ходов, номинальным — не более двух и меньше 5 %, всё остальное исключается.

5. **Недостаточная мощность бинарных конечных точек.** Перед этой серией мы вычислили, что обнаружение различия в доле решённых задач требует примерно 72 задачи при измеренной частоте несовпадений, и заявили об этом, вместо того чтобы приводить нуль как эквивалентность. Текущая серия покрывает это требование, поэтому §7 вообще делает утверждение о способности решать задачи.

12.2 Два отзыва

Оба приводятся, потому что отзыв — это тоже результат об измерении.

«Цензурирование по лимиту времени объясняет разрыв в способности»: отозвано; цензурирования никогда не существовало. Это было последующее утверждение от дефекта детектора из пункта 3. После исправления детектора истинное число цензурированных по времени запусков в корпусе оказалось нулевым. Последующее наблюдение о том, что повышение лимита восстановило четыре из семи провалившихся задач, затем было проверено против измеренной базовой частоты восстановления вердикта между неизменёнными повторными запусками (5/11); вероятность увидеть четыре или более восстановлений при одной этой базовой частоте составляет 0.40^{live}. Ничем не примечательно, отозвано.

«Только-состояние менее стабильно»: отозвано на третьем испытании. После двух испытаний рука только-состояния показывала более высокую дисперсию между испытаниями на 10 из 12 задач ($p = 0.039$). Третье испытание переместило это к 8 из 12 ($p = 0.388$). Утверждение умерло ровно так, как и следует ожидать от двухиспытательного p чуть ниже 0.05. Остановись серия на двух испытаниях, в эту статью попал бы ложный эффект.

Чего отзывы не затронули. Каждое измерение стоимости, токенов и кэша в этой статье не зависит от вердиктов, а результаты по способности решать задачи из §7 происходят из дизайна, который появился позже всего вышеперечисленного и покрывает требование к мощности, которое его мотивировало.

12.3 Промежуточный случай контаминации, сохранённый как предупреждение

Ранний анализ схемы в этом проекте дал $p = 7.9 \times 10^{-8}$ в пользу сильного эффекта. При повторном запуске на карантинно-очищенном корпусе с тем же скриптом то же сравнение даёт $p = 9.2 \times 10^{-2\mathsf{live}}$. Контаминированные ячейки были сконцентрированы в сравниваемых руках: арифметика была верна, а вводила в заблуждение именно шапка столбца.

13 Связанные работы

Память со структурированным состоянием. skill.state [Badhe et al., 2026] — это проверяемая система, описанная в §2. Семейство, к которому она принадлежит (замена истории поддерживаемым моделью структурированным представлением), разделяет предположение, которое мы фальсифицируем: что представление — это достаточная статистика для оставшейся работы.

Детерминированное состояние выполнения. ledger [Wang et al., 2026] поддерживает объективную запись, выводимую из завершённых взаимодействий, и добавляет её к иначе немодифицированному агенту, сохраняющему свою историю. Наше исследование композиции (§10) варьирует границу между двумя дизайнами. Условия различаются так, что стоит сказать это прямо: ledger добавляет детерминированный слой к агенту с полной историей, тогда как мы спрашиваем, что должен содержать детерминированный слой, когда история удалена.

Управление контекстом сжатием. Рантаймы суммаризации, скользящие окна и обученное сжатие промпта [Jiang et al., 2023] атакуют ту же проблему роста, сжимая транскрипт вместо его замены. Контрольные группы оригинала с подобранным бюджетом сообщают, что при равном размере промпта они набирают значительно ниже структурированного состояния^{orig}, что служит свидетельством в пользу того, что выигрыш несёт структура, а не краткость — вывод, которому наши результаты не противоречат. Наша находка о том, что теряется при полном отбрасывании транскрипта, а не о том, насколько хорошо его можно сжать.

Компоненты харнеса по отдельности. Fan et al. [2026] запускают 176 сопоставленных настроек, независимо варьирующих планирование, пространство действий и управление контекстом внутри одного фиксированного цикла выполнения, на четырёх моделях на SWE-Bench Verified и Terminal-Bench 2.1. Их условие контекста с исключением-и-вызовом — ближайший опубликованный аналог нашей оси хвоста (§7): оба урезают транскрипт и дают агенту механизм вернуть то, что было урезано. Сообщается, что их канал вызова добавляет мало, как только исключение установлено, что зеркалит нашу собственную находку: счётчики покрытия `verified_*` плагина тождественно нулевые в каждом запуске серии композиции (§10) — канал, построенный для проверки того, что ничто в 435 запусках не задействует. Мы читаем это как один и тот же режим отказа, достигнутый из двух разных дизайнов, а не как совпадение названий.

Слепота к контексту. Xu et al. [2026] утверждают, что модель не может надёжно вывести размер блока, недавность или оставшийся бюджет только из промпта, и вводят видимый канал внутреннего состояния, чтобы агент мог видеть, чем управляет. Наш результат о механизме (§8) согласуется с этим диагнозом, не нуждаясь в обращении к нему: неуспешный запуск только-состояния тратит большинство своих команд на повторный вывод уже произведённых фактов, что как раз и предсказывает слепота к контексту для агента, недостаточно информированного о собственной истории. Их исправление работает с

представлением модели о своём контексте; наше (§10) работает с тем, кто изначально пишет запись. Они не конкурируют, и §14 называет их комбинацию направлением, которое мы не измеряли.

Не теряющий данных рекурсивный контекст. Ehrlich and Blackman [2026] поддерживают детерминированное дерево иерархических сводок с не теряющими данных указателями обратно к исходному тексту, позволяя агенту рекурсивно углубляться в узел сводки, не теряя возможность извлечь то, что было сведено. Это то же самое базовое утверждение, что и у ledger в нашей постановке (детерминированный слой может нести информацию, которую собственная свободная сводка модели отбросила бы), применённое рекурсивно, а не как один дополнительный слой. Это указывает, что граница, изучаемая нами в §10 — написанное моделью против написанного рантаймом состояние — не единственная доступная ось; детерминированный слой также может варьироваться по глубине.

Агентные циклы рассуждения. Базовая линия транскрипта в стиле ReAct [Yao et al., 2023]: чередующиеся рассуждение и действие с только дополняемой историей. Её профиль стоимости при кэшировании префикса — то, что делает сравнение в §9 неочевидным, поскольку растущий промпт в основном обслуживается кэшем.

Бенчмарки с программными верификаторами. Мы используем Terminal-Bench, чьи задачи несут собственные верификаторы, по причине, данной в §5: вердикт не должен зависеть от суждения модели о собственном выводе. InterCode CTF [Yang et al., 2023] и τ-bench [Yao et al., 2024] используются оригиналом и разделяют это свойство.

Методология оценки. Наш методологический вклад (§12) применяет стандартную практику воспроизводимости (внешние вердикты, предрегистрированная мощность, письменные отзывы) к жанру, где, по нашему опыту, доминирующая ошибка носит определительный, а не статистический характер: столбец, помеченный как свойство метода, на самом деле измеряет харнес.

14 Заключение

Память со структурированным состоянием мотивирована верным наблюдением: неограниченный транскрипт — плохая рабочая память, и его стоимость растёт со всем, что было прежде. Опубликованная оценка skill.state показывает, что поддерживаемый моделью блок состояния ограничивает рост промпта почти постоянным размером, сохраняя качество, и на своём собственном бенчмарке этот результат воспроизводится, в том числе при учёте кэша.

На реальных программных задачах он не держится. По 1044 запускам предрегистрированного дизайна метод в опубликованном виде решает 0.349 задач против 0.613 у контрольной группы с транскриптом, проигрывая 82 парных задач и выигрывая 13 ($p = 2.1 \times 10^{-13}$), причём дефицит расширяется на длинных горизонтах, где механизм должен окупать себя. Восстановление шести дословных обменов рядом с блоком состояния полностью устраняет различие, что помещает ущерб в отброшенную историю, а не в абстракцию состояния.

Причина, по которой две постановки расходятся, структурна, а не эмпирична. Синтетический бенчмарк фиксирует число обращений к модели за эпизод, так что агент, отбросивший то, что позже понадобится, ничего не может потратить на восстановление этого. Реальные агенты могут и делают это: в неуспешных запусках только-состояния 58 % всех команд — это повторы уже выполненного осмотра, против 20 % у контрольной группы. Это тезис статьи в одном предложении: ограничение промпта не ограничивает информацию, требуемую задачей; разница оплачивается через цикл инструментов, куда не заглядывает ни одна метрика размера промпта.

Экономика следует из того же механизма и инвертирует привычное прочтение. При равном исходе рука только-состояния несёт половину пикового промпта контрольной группы и стоит 2.57× дороже, потому что поддержание состояния тарифицируется по выходному тарифу, а блок, переписываемый на каждом ходу, теряет кэш префикса, делающий только дополняемый транскрипт дешёвым. Измеренные на токен, руки состояния выглядят эффективными; измеренные на решённую задачу на доллар по всей серии, опубликованная конфигурация — худшая из четырёх. Однако на сравнении при совпадающем исходе, которое эта статья трактует как честное, каждая конфигурация состояния, включая лучшую, стоит дороже контрольной группы (1.15× при $k{=}6$, $p = 0.04$).

Практический вывод поэтому не в том, что от структурированного состояния следует отказаться, а в том, что его не следует использовать для замены недавней истории. Конфигурация, выигрывающая на этом корпусе, сохраняет и то, и другое, и открытый вопрос, который мы измеряем далее, — что должно содержать поддерживаемое рантаймом состояние, чтобы оставшийся разрыв закрылся (§10).

Лестница композиции из §10 останавливается на v7 (записи о том, что пробовали и что не сработало), потому что там останавливаются наши собственные запуски; это не то место, где останавливается пространство дизайна. Два механизма, всплывшие в §13, находятся на один шаг дальше, и ни один не был запущен на этом корпусе. Канал проприоцепции VISTA [Xu et al., 2026] дал бы агенту явный сигнал о том, что уже содержит написанное рантаймом состояние, что прямо связано с повторным выводом, который мы измеряем в §8: блок состояния, форму которого агент не видит, — это блок состояния, который он будет перепроверять. Рекурсивные, не теряющие данных указатели LCM [Ehrlich and Blackman, 2026] позволили бы ступени суммировать собственную историю вместо того, чтобы только накапливать её, решая тот единственный режим отказа, который лестница не может исправить добавлением новых полей: неограниченный рост самого блока состояния на высоких ступенях. Ступень, сочетающая оба подхода — написанное рантаймом состояние, форму которого агент может запросить, рекурсивно суммированное без потери возможности извлечь то, что было сведено, — естественная v8 и наиболее прямая проверка того, продолжается ли подъём лестницы или он уже вышел на плато к v6. Мы её не строили и не запускали; здесь она названа как следующее измерение, а не заявлена как результат.

Методологический вывод, возможно, переживёт эмпирический. В этом жанре различие между находкой и артефактом редко заключается в модели, методе или бенчмарке. Оно в учёте: в какой валюте измеряется стоимость, какую руку штрафует ограничение харнеса и по-прежнему ли шапка столбца описывает то, что в столбце содержится.

Список литературы

Sanket Badhe, Priyanka Tiwari, and Jonghyun Chung. SKILL.state: Scalable long-horizon agent skills. arXiv preprint arXiv:2608.26263, 2026. v3, 02 Sep 2026.

Clint Ehrlich and Theodore Blackman. LCM: Lossless context management. arXiv preprint arXiv:2605.04050, 2026.

Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An empirical study of harness design for coding agents. arXiv preprint arXiv:2609.20804, 2026.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In Proceedings of EMNLP, 2023.

Zehao Wang, Yisen Xu, Chenglin Li, Chao Peng, Bram Adams, Ahmed E. Hassan, and Tse-Hsun Chen. Turning interaction history into execution state: A runtime layer for long-horizon coding agents. arXiv preprint arXiv:2608.00808, 2026.

Binyan Xu, Haitao Li, and Kehuan Zhang. LLM agents are latent context managers: Eliciting self-managed context via state proprioception. arXiv preprint arXiv:2606.30005, 2026.

John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. InterCode: Standardizing and benchmarking interactive coding with execution feedback. In Advances in Neural Information Processing Systems, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In International Conference on Learning Representations, 2023.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045, 2024.

A Воспроизводимость

Набор данных. 1044 запусков, одна строка на запуск, с происхождением по каждой строке: идентификатор запуска, временные метки начала и завершения, ревизия репозитория, хэш содержимого плагина, хэш схемы, модель, испытание, условие, реализованное значение k, вердикт верификатора, причина остановки, число ходов и вызовов, три числа токенов по отдельности, пиковый промпт и производная стоимость. Каждая строка была перекрёстно проверена поле за полем против сырого вывода сборщика с нулевыми расхождениями[live].

Регенерация чисел статьи. Каждая таблица, каждая координата рисунка и каждое встроенное число в этой статье производится одним детерминированным анализом из опубликованных данных запусков. Ни одно измеренное число не записывается в исходник рукописи вручную; текст ссылается на сгенерированные определения, так что изменение данных не может оставить устаревшее число в прозе.

Анализ дополнительно применяет гейт целостности, сформулированный как Алгоритм 3: он пересчитывает опубликованные ключевые значения из сырых строк и отказывается выдать документ, если хотя бы одно из них расходится. Гейт был проверен по построению: искажение выборки вердиктов на входе приводит к его сбою с указанием каждой затронутой величины[live].

Algorithm 3 Гейт целостности на этапе сборки

Require: каноническая таблица запусков D; опубликованные ключевые значения V

пересчитать каждую таблицу и встроенную величину из D
$M \leftarrow$ пересчитанные ключевые значения
for каждого $v \in V$ **do**
    if $M[v] \neq V[v]$ **then**
        сообщить о v с обоими значениями
        прервать сборку
    end if
end for
выдать таблицы, данные рисунков и определения макросов
типографировать документ

Точный вызов находится в инструкциях по сборке репозитория, а не здесь.

Статистика. Точные тесты Макнемара и знаковый тест вычисляются из биномиального распределения в замкнутой форме; интервалы Уилсона используются для долей; тренд по горизонту использует ранговую корреляцию Спирмена с перестановочным p-значением при фиксированном сиде. Реализация зависит только от стандартной библиотеки, поэтому выполняется без изменений на измерительном хосте.

Исключения, с подсчётами. Попытки, не вернувшие вердикт верификатора, записываются отдельно и исключаются; подавляющее большинство было переставлено в очередь и позже закрыто с реальным вердиктом, а остаток симметричен по рукам. Запуски, измеренные в условиях насыщения провайдера, карантинируются по заранее объявленному правилу из §12. Там, где перезапуск оркестратора выполнил одну ячейку дважды, билдер сохраняет самый ранний запуск — правило, зафиксированное до того, как был прочитан хоть один вердикт, чтобы повтор не мог продвинуть себя, выиграв[code].

Стек. Версии агента, харнеса и плагина зафиксированы и записаны для каждого запуска; хэши содержимого для плагина и схемы записаны в каждую строку, так что запуск можно привязать к точному коду, который его произвёл.

Артефакты. Набор данных, сырой вывод сборщика, анализ и исходник рукописи публикуются вместе. [**NOTE:** DOI Zenodo будет присвоен при подаче; URL репозитория будет заполнен, когда релиз станет публичным]

B Пробелы отчётности в исходной оценке

Составлено во время полного извлечения исходной статьи, каждый пункт проверяем напрямую по ней[orig]. Они записаны здесь, потому что несколько из них — причина, по которой воспроизведение вынуждено делать собственный выбор, и читатель должен знать, какой выбор был вынужденным.

1. **Схемы.** Дословный список полей дан для одного из четырёх доменов оценки. Для двух других схема выводима только из единственной примерной траектории; для четвёртого не дано никакой. Нигде не описан метод проектирования схемы, и нет анализа чувствительности. Воспроизводящему поэтому приходится изобретать схему, что само по себе источник расхождения, и мотивация трактовать проектирование схемы как фактор, а не как константу.

2. **Кэширование и деньги.** Полнотекстовый поиск даёт ноль вхождений слов cache, billing или иной денежной терминологии. Все экономические утверждения — в логических числах токенов. Статья молчит по этому вопросу, а не кратка.

3. **Дисперсия и повторения.** Таблицы устойчивости к шуму и восстановления состояния приводят точечные значения без указанного стандартного отклонения или числа повторений.

4. **Контроль сжатия недоопределён.** Базовая линия обученного сжатия приводится без коэффициента сжатия или варианта. Бюджет усечения указан в символах в одном месте и в токенах в другом.

5. **Статистическое утверждение не показано.** Значимость утверждается как единый парный t-тест при $p < 0.01$ для горизонтов ≥ 50, без значений по отдельным сравнениям и без поправки на множественные сравнения.

6. **Незамеченный нуль.** В одной конфигурации с открытыми весами на самом длинном приводимом горизонте рантайм со структурированным состоянием и стабильная базовая линия набирают идентично. Таблица это содержит; текст это не обсуждает.

7. **Авторитет над числом шагов.** Горизонт — это подсчёт внешне генерируемых событий, и ни одно условие не варьирует, кто контролирует число шагов. Это свойство, анализируемое в §6, и оно — единственное наиболее значимое различие между той оценкой и нашей.